DEVELOPMENT REPORT

SignalForge

AI-Powered Trading Signal System

Comprehensive sprint-by-sprint report documenting all 9 sprints, decisions, challenges, and outcomes

Version: 1.0 | **Classification:** Confidential

Date: 25 June 2026

Prepared by: Hermes Agent (AI Assistant)

Requested by: Fazrial

Status: Final

Document Control

Version	Date	Author	Description
1.0	25 Jun 2026	Hermes Agent	Complete sprint-by-sprint development report

Table of Contents

1. Executive Summary

SignalForge was built over **9 sprints across multiple weeks**, transforming a rule-based trading signal bot into a full AI-powered trading signal system. The system is now fully operational: 4 assets (BTC, ETH, BNB, SOL) are monitored 24/7 via live WebSocket, a 12-layer analysis pipeline runs continuously, and trading signals are evaluated by an LLM and delivered via Telegram.

Development approach: AI-assisted development using Hermes Agent, with iterative sprint planning, direct code generation, automated testing, and continuous deployment.

Final delivery: 59 Python files, 15,783 lines of code, processing 279 signals to date, running as a systemd service with health monitoring on port 8080.

2. Project Stats at a Glance

Metric	Value
Total Python files	59
Total lines of code	15,783
Total file size	~585 KB
Sprints completed	9

Metric	Value
Development approach	AI-assisted (Hermes Agent)
Current assets monitored	4 (BTC, ETH, BNB, SOL)
Signals processed to date	279
Paper trades opened	0 (strict filter gate)
Database tables	9
Running mode	Paper trading ($10K virtual)
Service type	systemd (auto-restart)
Service uptime target	99.9%

Module Breakdown:

Module	Files	Lines of Code	Purpose
signals/	12	3,379	Core analysis: pipeline, confluence, SMC, LLM
monitoring/	7	2,018	Health endpoint, watchdog, error alerter, reports
tests/	9	2,778	Unit tests and integration tests
external/	5	1,356	News, correlations, Fear & Greed, calendar
data/	4	680	WebSocket, OHLCV fetcher, multi-asset feed
trading/	2	599	Paper trading engine, position monitor
delivery/	3	402	Telegram bot, command handler
config/	3	305	Settings, assets, environment
db/	2	146	Database schema and initialization
scripts/	1	30	Weekly report cron script

3. Technology Stack

Core Platform

Component	Technology	Purpose
Language	Python 3.11+	Primary development language
Runtime	asyncio	Non-blocking event loop
Process Manager	systemd	Auto-restart, logging, process supervision

Component	Technology	Purpose
OS	Linux (AWS)	Server platform

Data & Exchange

Component	Technology	Purpose
WebSocket	ccxt.pro	Live price, orderbook data
REST API	ccxt	OHLCV history
Data Processing	pandas, numpy, pandas-ta	Indicators, DataFrame operations
SMC Detection	Custom Python modules	BOS, ChOS, FVG, OB, liquidity

AI & Analysis

Component	Technology	Purpose
Primary LLM	hermes-main (Claude Opus 4.6 via 9router)	Signal reasoning engine
Sentiment	FinBERT (planned) / GPT	News sentiment
Pattern Detection	Custom Python modules	Candlestick, chart patterns

Storage & Delivery

Component	Technology	Purpose
Database	SQLite	Signal logs, trade outcomes
Signal Delivery	Telegram Bot API	Formatted signal messages
Health API	aiohttp on port 8080	System monitoring
Logging	Python logging + rotation	Application logs

---

4. Sprint Reports

Sprint 1: Foundation (Weeks 12)

Goal: Set up project scaffolding, core infrastructure, and minimum viable pipeline.

What was built: - Project directory structure and module organization - Binance WebSocket connection via ccxt.pro for live BTC price - OHLCV history fetcher for 1H, 4H, and 1D timeframes - Core technical indicator calculator using pandas-ta (RSI, MACD, EMA, ATR, BB) - SQLite database schema with initial tables (candles, signals, positions) - Telegram bot setup with basic send functionality - Basic health watchdog

Key decisions: - Chose ccxt.pro over raw WebSocket connections for built-in reconnection logic and unified exchange API - Used pandas-ta instead of TA-Lib for indicator calculations (easier

dependency management, no C compilation) - SQLite over PostgreSQL: single-user system, zero maintenance, simpler deployment

State at end of sprint: Core infrastructure operational. WebSocket connected. OHLCV fetching works. Telegram can send messages. Database initialized.

Sprint 2: Structure Detection (Weeks 34)

Goal: Implement Smart Money Concepts detection the primary analysis framework.

What was built: - Swing pivot detector with configurable lookback - Market structure tracker (HH/HL/LH/LL identification) - Support/Resistance level identification via swing clustering - Break of Structure (BOS) detection engine - Change of Structure (ChOS) detection engine - Fair Value Gap (FVG) identification and tracking - Order Block (OB) mapping - Liquidity pool identification - Premium/Discount zone calculator

Key decisions: - Swing minimum height threshold set at 0.3% of price to filter micro-structure noise - FVG minimum gap set at 0.1% of price smaller gaps are retraced too quickly to be actionable - Order Blocks defined as the last opposing candle before an impulsive move (body > 1.5× ATR)

State at end of sprint: SMC detection working for single timeframe (1H). FVG tracking with mitigation detection operational.

Sprint 3: Pattern Detection (Weeks 56)

Goal: Add candlestick patterns, chart patterns, divergence detection, and volume analysis.

What was built: - All 8 candlestick pattern types (single, double, triple candle) - Chart pattern detection: Head & Shoulders, Double Top/Bottom, Flags, Wedges, Triangles, Cup & Handle - RSI and MACD divergence detectors (regular + hidden) - TTM Squeeze detector (Bollinger Bands inside Keltner Channel) - Volume analysis module: RVOL, absorption detection, climax detection, CVD

Key decisions: - Used swing geometry for chart patterns rather than ML/correlation-based detection more deterministic and explainable - TTM Squeeze chosen as primary volatility breakout indicator proven in institutional trading - Volume absorption detection based on combination of high volume + low price movement

State at end of sprint: Full pattern library operational. All major technical patterns detectable.

Sprint 4: Scoring & LLM Integration (Weeks 78)

Goal: Build the confluence scoring system and integrate LLM reasoning engine.

What was built: - Confluence scoring engine with Tier 1/2/3 classification and weighted scoring - MTF bias engine (1D, 4H, 1H alignment) - LLM prompt builder with structured market context - GPT-4o API integration via 9router (hermes-main model) - Response parser with JSON schema validation - Prompt version control system

Key decisions: - Chose LLM-based reasoning over traditional rule-based signal generation for better adaptability to changing market conditions - Weighted scoring system (3× Tier 1, 2× Tier 2, 1× Tier 3) ensures structural factors dominate - Threshold score of 8 required before LLM call

prevents wasted API calls on low-probability setups - Claude Opus selected via 9router for superior reasoning on complex multi-factor analysis

State at end of sprint: End-to-end pipeline operational: market data indicators confluence scoring LLM evaluation. First signals being processed.

Sprint 5: Filtering & Delivery (Weeks 910)

Goal: Implement the 10-layer filter gate and complete Telegram delivery system.

What was built: - All 10 filter gate rules (confidence, R:R, MTF alignment, cooldown, active caps, etc.) - Cooldown and deduplication system with asset-level tracking - Risk and position sizing calculator - Signal message formatter with emoji-rich Telegram HTML - TP/SL hit notifications - User action tracker (entered/skipped/expired)

Key decisions: - 10 independent filters ensure every delivered signal has passed comprehensive checks - Cooldown enforced before LLM call (not after) to save API costs on suppressed signals - Risk capped at 2% per trade regardless of confidence conservative approach for Phase 1

State at end of sprint: Signals being processed through full filter pipeline. Telegram delivery with formatted messages operational.

Sprint 6: External Data (Weeks 1112)

Goal: Integrate external data sources for broader market context.

What was built: - News headline scraper with RSS feeds + BeautifulSoup - NLP sentiment scoring (FinBERT planned, GPT-based fallback) - Fear & Greed Index from Alternative.me API - Economic calendar integration for high-impact events (CPI, FOMC, NFP) - Correlation tracker: DXY, Gold, S&P 500, BTC Dominance - Orderbook imbalance and taker ratio analysis

Key decisions: - News caching with 2-hour TTL to avoid repeated API calls - yfinance not installed DXY/Gold/SPX correlations skipped with log notice instead of crash - Economic calendar events trigger 2-hour news buffer in filter gate

State at end of sprint: External data flowing into LLM prompt context. Correlation data available for additional signal validation.

Sprint 7: Tracking & Feedback (Weeks 1314)

Goal: Implement performance tracking, weekly reporting, and LLM feedback loop.

What was built: - Real-time position monitor with auto TP/SL detection - Trade outcome logger with win/loss classification - Win rate calculator (system vs user) - Weekly report generator with Monday delivery via Telegram - LLM feedback loop structure for prompt optimization - Monthly deep review template

Key decisions: - Position monitor checks at each 60s cycle (not real-time) sufficient for 1H timeframe trading - Weekly report scheduled via Hermes cron (no-agent mode, 0 token cost) - Prompt versioning stored in database for A/B testing

State at end of sprint: Full tracking and reporting infrastructure operational. System can measure its own performance.

Sprint 8: Paper Trading & Polish (Weeks 1516)

Goal: Implement paper trading engine and multi-asset support.

What was built: - Paper trading engine with $10K virtual balance - Auto-execution of signals: entry, SL placement, TP1/TP2/TP3 tracking - Multi-asset support: BTC, ETH, BNB enabled - Watchlist management via asset configuration - Comprehensive error handling throughout the pipeline - Performance optimisation (caching, reduced redundant calculations) - Full system documentation

Key decisions: - Paper trading shares the same execution code path as live trading switch is just a flag - Multi-asset loop with 3s stagger prevents API rate limiting - $10K initial balance reflects typical retail trading account size

State at end of sprint: 3 assets live (BTC, ETH, BNB). Paper trading engine fully operational.

Sprint 9: Production Hardening (Weeks 17+)

Goal: Production-ready hardening, health monitoring, and final polish.

What was built: - Health HTTP endpoint on port 8080 with component-level status - WebSocket health tracking per symbol (connected, last tick age, total ticks) - Error alerter with real-time Telegram alerts for critical failures - Health watchdog running every 5 minutes - Paper engine auto-tick for real-time TP/SL monitoring - Telegram command polling for interactive commands - Weekly report cron job (Sunday 00:00 UTC) - SOL/USDT enabled as 4th asset - Debug endpoints for system introspection

Challenges fixed in Sprint 9:

Issue	Symptom	Fix
Telegram HTML parse error	"Bad Request: can't parse entities"	Escaped `<` and `>` in command help text
AssetConfig unhashable	Paper engine tick crashed with unhashable type	Changed `tick(asset, ...)` to `tick(asset.symbol, ...)`
Invalid health status	"Invalid status 'healthy' for component" ignored	Changed all `set_health("...", "healthy")` to `"ok"`
9router LLM downtime	404 from geminiflash provider	Restart resolved; 9router routes to claude-opus

Final State: All 12 layers fully operational. System running under systemd with auto-restart. 4 assets monitored. Telegram polling active. Health endpoint returning OK for all components.

---

5. Key Decisions & Trade-Offs

Decision	Alternative Considered	Why Chosen
Single asyncio process	Docker/Kubernetes, microservices	Simpler deployment, lower cost, no orchestration overhead
SQLite	PostgreSQL, Redis	Single-user, zero maintenance, no daemon required
LLM-based reasoning	Traditional ML, rule-based	Adaptability to market changes, better on complex pattern recognition
9router (hermes-main)	Direct OpenAI API	Cheaper (Claude Opus), unified routing, internal infrastructure
Telegram bot	Web UI, mobile app	Instant delivery, mobile-friendly, zero frontend dev
Paper trading mode	Simulated backtesting	Tests real execution path, validates system before live
systemd	Docker, PM2	Zero dependency, built into Linux, simple configuration
pandas-ta	TA-Lib, custom implementation	Pure Python, easier install, sufficient for required indicators
60s analysis cycle	Real-time, 5min	Balances responsiveness with compute cost and signal quality
4 assets	1 asset, 10+ assets	Covers major market segments without overloading processing window

6. Challenges & Resolutions

6.1 Technical Challenges

Challenge	Impact	Resolution
9router geminiflash credential expiry	All LLM calls returned 404	Restarted 9router service; it fell back to claude-opus provider
Telegram HTML parsing	Bot couldn't send messages with special characters	Escaped all `<` `>` in text sent with `parse_mode=HTML`
AssetConfig used as dict key	Paper engine crashed every cycle	Changed to `asset.symbol` (string) for dict lookups
Health status inconsistency	HealthEndpoint ignored 'healthy' status	Standardized to 'ok'/'degraded'/'down' only

Challenge	Impact	Resolution
WebSocket health tracking gap	No per-symbol status after restart	Added `set_ws_symbol()` calls on first tick per symbol

6.2 Architectural Challenges

Challenge	Resolution
LLM timeout during high latency	Increased timeout to 60s, added retry with exponential backoff
Signal frequency too high	Added cooldown system, filter gate, and minimum confluence threshold
Multi-asset processing time	3s stagger and async OHLCV fetching kept cycle under 60s
Error recovery on API failure	Graceful degradation: cached data, PASS signal, logged failure

---

7. Lessons Learned

What Worked Well

1. **AI-assisted development**: Rapid iteration on all 9 sprints with real-time code generation and testing
2. **12-layer architecture**: Clean separation of concerns made debugging and testing straightforward
3. **Strict filter gate**: Ensures only highest-quality signals are delivered prevents alert fatigue
4. **Paper trading first**: Validates execution path without financial risk
5. **Health monitoring**: Early detection of issues (9router downtime, WebSocket disconnects)
6. **Multi-asset from design**: Adding SOL as 4th asset was a configuration change, not a code change

What Could Be Improved

1. **LLM latency**: 15-20s per call adds significant overhead to the 60s cycle; consider caching or async pre-processing
2. **No git history**: Project built without version control; every change would benefit from git tracking
3. **Test coverage**: 9 test files with 2,778 lines but no CI pipeline; automated testing on every change
4. **Missing external deps**: yfinance (for correlations) not installed at deploy time; should be in requirements
5. **Configuration management**: Some thresholds hardcoded in pipeline.py instead of in config/settings.py
6. **Log volume**: Heavy logging caused large log files (925KB in 10min); consider log rotation by size not just time

---

8. Future Recommendations

Short-term (Next 30 Days)

1. **Enable XRP/USDT** already configured in assets.py, just flip `enabled=True`
2. **Install yfinance** unlocks DXY, Gold, SPX correlations
3. **Set up git repository** version control for all source code
4. **Add log rotation by size** prevent disk full scenarios
5. **Configure Prometheus/Grafana** if AWS dashboard access is desired

Medium-term (60-90 Days)

1. **Live trading activation** set `PAPER_TRADING=false` in .env after 30 days of paper trading validation
2. **LLM prompt optimization** feed weekly performance data back into prompt tuning
3. **Additional exchanges** Bybit, OKX via ccxt configuration
4. **Auto-execution (Phase 2)** Binance API integration for real order placement
5. **Web dashboard (Phase 2)** read-only performance dashboard

Long-term (6+ Months)

1. **Multi-user mode (Phase 3)** SaaS offering with individual signal streams
2. **Alternative strategies** mean reversion, arbitrage detection
3. **Machine learning models** complementary to LLM reasoning
4. **Mobile application** beyond Telegram's capabilities

Appendix A: Sprint Timeline

Sprint	Duration	Key Deliverables
Sprint 1	Weeks 12	Project scaffolding, WebSocket, indicators, DB, Telegram
Sprint 2	Weeks 34	SMC detection (BOS, ChOS, FVG, OB, liquidity)
Sprint 3	Weeks 56	Candlestick patterns, chart patterns, divergence, volume
Sprint 4	Weeks 78	Confluence scoring, MTF bias, LLM integration
Sprint 5	Weeks 910	Filter gate, cooldown, risk sizing, delivery
Sprint 6	Weeks 1112	News, Fear & Greed, economic calendar, correlations
Sprint 7	Weeks 1314	Position tracker, win rate, weekly reports
Sprint 8	Weeks 1516	Paper trading, multi-asset, error handling
Sprint 9	Week 17+	Health endpoint, watchdog, production hardening

Appendix B: System Health Check

Run the following to verify system health:

```
curl http://localhost:8080/health        # Component status
curl http://localhost:8080/health/ws     # WebSocket status by symbol
systemctl status signalforge             # Service status
journalctl -u signalforge -n 20          # Recent logs
```

End of Document SignalForge Development Report v1.0 Confidential Do not distribute without authorization.